



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.
Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Отчет по лабораторной работе по дисциплине «Операционные системы»

Тема Обработчик прерывания от системного таймера в системах разделения времени

Студент Павлов Д.В.

Группа ИУ7-53Б

Преподаватель Рязанова Н.Ю.

Москва, 2025

СОДЕРЖАНИЕ

1	Функции обработчика прерывания от системного таймера	3
1.1	UNIX	3
1.2	Windows	4
2	Пересчет динамических приоритетов	5
2.1	UNIX	5
2.2	Windows	7
	ВЫВОДЫ	14
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	15

1 Функции обработчика прерывания от системного таймера

1.1 UNIX

По тикку:

- инкремент счетчика реального времени;
- инкремент счетчика времени, прошедшего с момента запуска системы;
- инкремент счетчика процессорного времени, полученного текущим процессором в режиме задачи и в режиме ядра;
- декремент счетчиков времени, оставшегося до отправления на выполнение отложенных вызовов, при достижении нулевого значения счетчиков выставление флага, указывающего на необходимость запуска обработчика отложенного вызова
- декремент кванта.

По главному тикку:

- выполнение связанных с планированием процессов, например пересчет приоритетов и проверку истечения временного кванта для процесса;
- пробуждение (регистрация отложенного вызова процедуры *wakeup*, которая перемещает дескриптор процесса из списка «спящих» в очередь «готовых к выполнению») системных процессов, таких как *swapper* и *pagedaemon*;
- декремент счетчика времени, оставшегося до отправки одного из следующих сигналов:
 - **SIGALRM** — сигнал посылается процессу по истечении времени, которое было предварительно задано через системный вызов *alarm*;
 - **SIGPROF** — сигнал, посылаемый процессу по истечению времени, заданного в таймере профилирования;
 - **SIGVTALRM** — сигнал, посылаемый процессу по истечении времени, заданного с помощью системного вызова *setitimer*.

По кванту:

- посылка сигнала **SIGXCPU** активному процессу, если тот превысил выделенный для него квант процессорного времени [1].

1.2 Windows

По тикку:

- инкремент счетчика реального времени;
- декремент счетчика времени отложенных задач;
- декремент кванта.

По главному тикку:

- Инициализация диспетчера настройки баланса путем сбрасывания объекта «событие», на котором он ожидает (диспетчер настройки баланса по событию от таймера сканирует очередь готовых процессов и повышает приоритет процессов, которые находились в очереди в состоянии ожидания дольше 4х секунд).

По кванту:

- инициализирует диспетчеризацию потоков с добавлением соответствующего объекта в очередь DPC [2].

2 Пересчет динамических приоритетов

2.1 UNIX

Планирование процессов в UNIX основано на приоритете процессов. Каждый процесс обладает приоритетом планирования, изменяющимся с течением времени. Планировщик всегда выбирает процесс с наивысшим приоритетом. Для диспетчеризации процессов с равным приоритетом применяется вытесняющее квантование времени. Изменение приоритетов процессов происходит динамически на основе количества используемого ими процессорного времени. Необходимо проводить пересчет приоритетов, чтобы исключить бесконечное откладывание.

Традиционное ядро UNIX является строго невытесняющим, однако в современных системах UNIX ядро является вытесняющим – то есть процесс в режиме ядра может быть вытеснен более приоритетным процессом в режиме ядра. Ядро сделано вытесняющим для того, чтобы система могла обслуживать такие процессы, как процессы на звуковой карте.

В операционных системах Unix был введен механизм многозадачности, при котором один процесс может включать несколько параллельно выполняющихся потоков. Эти потоки разделяют адресное пространство и другие ресурсы процесса, но выполняются независимо друг от друга. Каждый поток имеет свой контекст, который включает в себя регистры и стек, специфичные для его выполнения. Однако общие для всех потоков ресурсы остаются в контексте самого процесса.

Приоритет процесса задается любым целым числом, которое лежит в диапазоне от 0 до 127 (чем меньше число, тем выше приоритет)

1. 0 - 49 – зарезервированы для ядра (приоритеты ядра статические)
2. 50 - 127 – прикладные процессы (приоритеты прикладных процессов динамические, т.е. могут изменяться во времени)

Структура `proc`, представляющая собой дескриптор процесса, содержит следующие поля, относящиеся к приоритетам:

1. `p_pri` – текущий приоритет планирования;
2. `p_usrpri` – приоритет режима задачи;
3. `p_cpu` – результат последнего измерения использования процессора;

4. `p_nice` – фактор «любезности», который устанавливается пользователем.

Планировщик использует `p_pri` для принятия решения о том, какой процесс направить на выполнение. `p_pri` и `p_usrpri` равны, когда процесс находится в режиме задачи.

Значение `p_pri` может быть изменено (повышено) планировщиком для того, чтобы выполнить процесс в режиме ядра. В таком случае `p_usrpri` будет использоваться для хранения приоритета, который будет назначен процессу, когда тот вернется в режим задачи.

`p_cpu` инициализируется нулем при создании процесса (и на каждом тике обработчик таймера увеличивает это поле текущего процесса на 1, до максимального значения равного 127).

Ядро системы связывает приоритет сна с событием или ожидаемым ресурсом, из-за которого процесс может блокироваться (приоритет сна определяется для ядра, поэтому лежит в диапазоне 0–49). Когда процесс «просыпается», ядро устанавливает `p_pri`, равное приоритету сна события или ресурса, по которому произошла блокировка (значение приоритета сна для некоторых событий в системе 4.3BSD представлены в таблице 2.1).

Таблица 2.1 – Приоритеты сна в ОС 4.3BSD

<i>Приоритет</i>	<i>Значение</i>	<i>Описание</i>
PSWP	0	Свопинг
PSWP + 1	1	Страничный демон
PSWP + 1/2/4	1/2/4	Другие действия по обработке памяти
PINOD	10	Ожидание освобождения inode
PRIBIO	20	Ожидание дискового ввода-вывода
PRIBIO + 1	21	Ожидание освобождения буфера
PZERO	25	Базовый приоритет
TTIPRI	28	Ожидание ввода с терминала
TTOPRI	29	Ожидание вывода с терминала
PWAIT	30	Ожидание завершения процесса потомка
PLOCK	35	Консультативное ожидание блок. ресурса
PSLEP	40	Ожидание сигнала

Каждую секунду ядро системы инициализирует отложенный вызов процедуры `schedcpu()`, которая уменьшает значение `p_pri` каждого процесса исходя из фактора «полураспада» (в системе 4.3BSD считается по формуле 2.1)

$$decay = \frac{2 \cdot load_average}{2 \cdot load_average + 1}, \quad (2.1)$$

где *load_average* — это среднее количество процессов, находящихся в состоянии готовности к выполнению, за последнюю секунду.

Также процедура `schedcpu()` пересчитывает приоритеты для режима задачи всех процессов по формуле 2.2,

$$p_usrpri = PUSER + \frac{p_cpu}{2} + 2 \cdot p_nice, \quad (2.2)$$

где **PUSER** — базовый приоритет в режиме задачи, равный 50 [1].

Таким образом, если процесс в последний раз использовал большое количество процессорного времени, то его *p_cpu* будет увеличен => рост значения *p_usrpri* => понижение приоритета. Чем дольше процесс простаивает в очереди на выполнение, тем больше фактор полураспада уменьшает его *p_cpu* => повышение его приоритета. Такая схема предотвращает бесконечное откладывание низкоприоритетных процессов. Применение данной схемы предпочтительно процессам, осуществляющим много операций ввода-вывода, в противоположность процессам, производящим много вычислений. То есть динамический пересчет приоритетов процессов в режиме задачи позволяет избежать бесконечного откладывания.

2.2 Windows

В ОС семейства Windows процессу при создании назначается базовый приоритет. Относительно базового приоритета процесса потоку назначается относительный приоритет. Планирование осуществляется на основе приоритетов потоков, готовых к выполнению. Поток с более низким приоритетом вытесняется потоком с более высоким приоритетом, в тот момент когда этот поток становится готовым к выполнению. По истечении кванта времени, выделенного текущему потоку, ресурс передается самому приоритетному потоку в очереди готовых к выполнению. Каждый поток имеет динамический приоритет. Это приоритет, который планировщик использует для определения того, какой поток следует выполнить. Изначально динамический приоритет потока совпадает с базовым приоритетом процесса.

В ОС семейства Windows используется 32 уровня приоритета:

1. от 0 до 15 — динамические (изменяющиеся) уровни, при этом уровень 0 зарезервирован для потока обнуления страниц;

2. от 16 до 31 — статические уровни приоритета реального времени.

Динамический приоритет потока может временно повышаться или понижаться диспетчером планирования, чтобы улучшить отклик системы и обеспечить справедливое распределение процессорного времени. Однако система применяет такие корректировки только к потокам с базовым (статическим) приоритетом в диапазоне 0–15. Потоки с базовым приоритетом 16–31 считаются статическими: их приоритет не изменяется во время выполнения и используется исключительно для задач реального времени.

Уровни приоритета потоков назначаются с двух позиций: Windows API и ядра операционной системы. Windows API сортирует процессы по классам приоритета, которые были назначены при их создании:

- Реального времени – Real-time (4);
- Высокий – High (3);
- Выше обычного – Above Normal (7);
- Обычный – Normal (2);
- Ниже обычного – Below Normal (5);
- Простоя – Idle (1).

Затем назначается относительный приоритет потоков в рамках процес- сов:

- критичный по времени (time critical (15));
- наивысший (highest (2));
- выше обычного (above normal (1));
- ниже обычного (below normal (-1));
- обычный (normal (0));
- низший (lowest (-2));
- простой (idle (-15)).

Исходный базовый приоритет потока наследуется от базового приоритета процесса. Процесс по умолчанию наследует свой базовый приоритет у того процесса, который его создал.

Таким образом, в Windows API каждый поток имеет базовый приоритет, являющийся функцией класса приоритета процесса и его относительного приоритета процесса. В ядре класс приоритета процесса преобразуется в базовый приоритет. В таблице 2.2 приведено соответствие между приоритетами Windows API и ядра системы приоритета.

Таблица 2.2 – Соответствие между приоритетами *Windows API* и ядра Windows

	<i>real-time</i>	<i>high</i>	<i>above normal</i>	<i>normal</i>	<i>below normal</i>	<i>idle</i>
<i>time critical</i>	31	15	15	15	15	15
<i>highest</i>	26	15	12	10	8	6
<i>above normal</i>	25	14	11	9	7	5
<i>normal</i>	24	13	10	8	6	4
<i>below normal</i>	23	12	9	7	5	3
<i>lowest</i>	22	11	8	6	4	2
<i>idle</i>	16	1	1	1	1	1

В Windows также включен диспетчер настройки баланса, который сканирует очередь готовых процессов 1 раз в секунду. Если он обнаруживает потоки, ожидающие выполнения более 4 секунд, диспетчер настройки баланса повышает их приоритет до 15. Когда истекает квант, приоритет потока снижается до базового приоритета. Если поток не был завершен за квант времени или был вытеснен потоком с более высоким приоритетом, то после снижения приоритета поток возвращается в очередь готовых потоков.

Текущий приоритет потока в динамическом диапазоне (от 1 до 15) может быть изменён планировщиком вследствие следующих причин:

- повышение приоритета после завершения операций ввода-вывода;
- повышение приоритета владельца блокировки;
- повышение приоритета вследствие ввода из пользовательского интерфейса;

- повышение приоритета вследствие длительного ожидания ресурса исполняющей системы;
- повышение приоритета вследствие ожидания объекта ядра;
- повышение приоритета в случае, когда готовый к выполнению поток не был запущен в течение длительного времени;
- повышение приоритета проигрывания мультимедиа службой планировщика *MMCSS*.

Кроме того, в различных заголовочных файлах Windows указаны рекомендуемые значения, которые следует использовать коду режима ядра при вызове таких функций, как *KeSetEvent* и *KeReleaseSemaphore*. Эти значения определены как *EVENT_INCREMENT* и *MUTANT_INCREMENT* соответственно. В заголовках они всегда задаются равными 1, поэтому можно считать, что большинство случаев повышения приоритета при завершении ожидания на таких объектах приводит к увеличению приоритета на единицу.

Текущий приоритет потока в динамическом диапазоне может быть понижен до базового путем вычитания всех его повышений. В таблице 2.3 приведены рекомендуемые значения повышения приоритета для устройств ввода-вывода.

Таблица 2.3 – Рекомендуемые значения повышения приоритета.

<i>Устройство</i>	<i>Повышение приоритета</i>
Звуковая карта	8
Клавиатура, мышь	6
Сеть, почтовый слот, именованный канал, последовательный порт	2
Жесткий диск, привод компакт-дисков, параллельный порт, видеоустройство	1

Большинство описанных выше повышений приоритетов перестают действовать после завершения кванта или нескольких, выделенных процессу. На рисунке 2.1 представлен пример повышения и понижения приоритетов.

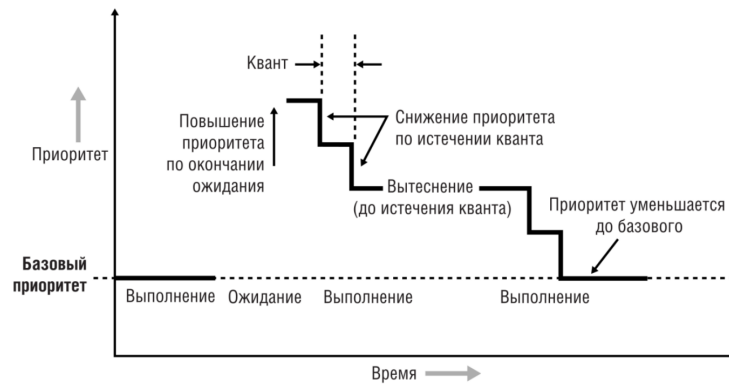


Рисунок 2.1 – Повышение и снижение приоритета

MMCSS Потоки, на которых выполняются различные мультимедийные приложения, должны выполняться с минимальными задержками, причем выполнение потоков, работающих с аудио должно быть наиболее приоритетным, так как человеческий слух чувствителен к задержкам. В Windows эта задача решается путем повышения приоритетов таких потоков драйвером **MMCSS** — MultiMedia Class Scheduler Service.

Одно из наиболее важных свойств для планирования потоков — категория планирования — первичный фактор определяющий приоритет потоков, зарегистрированных в **MMCSS**. Различные категории планирования представлены в таблице 2.4.

Таблица 2.4 – Категории планирования.

Категория	Приоритет	Описание
High (Высокая)	23-26	Потоки, связанные с обработкой аудио , работают с более высоким приоритетом, чем большинство потоков в системе, за исключением потоков, обеспечивающих выполнение критически важных системных задач.
Medium (Средняя)	16-22	Потоки, поддерживающие выполнение активных приложений, например, таких как Windows Media Player, имеют умеренный приоритет.
Low (Низкая)	8-15	Потоки, не относящиеся к приложениям первого плана или задачам повышенного приоритета, выполняются с обычным уровнем приоритета.
Exhausted (Исчерпавших потоков)	1-7	Потоки, использовавшие своё доступное время на процессоре, продолжают выполнение только тогда, когда потоки с более высоким приоритетом не готовы к работе.

Функции драйвера MMCSS временно повышают приоритет потоков, зарегистрированных с MMCSS до уровня, который соответствует категории планирования. Потом их приоритет снижается до уровня, соответствующего категории планирования Exhausted, для того, чтобы другие потоки тоже могли получить ресурс.

IRQL

Контроллеры прерываний сами устанавливают приоритетность своих прерываний, однако Windows определяет свою схему приоритетности, называемую IRQL, которая представлена на рисунке 2.2.

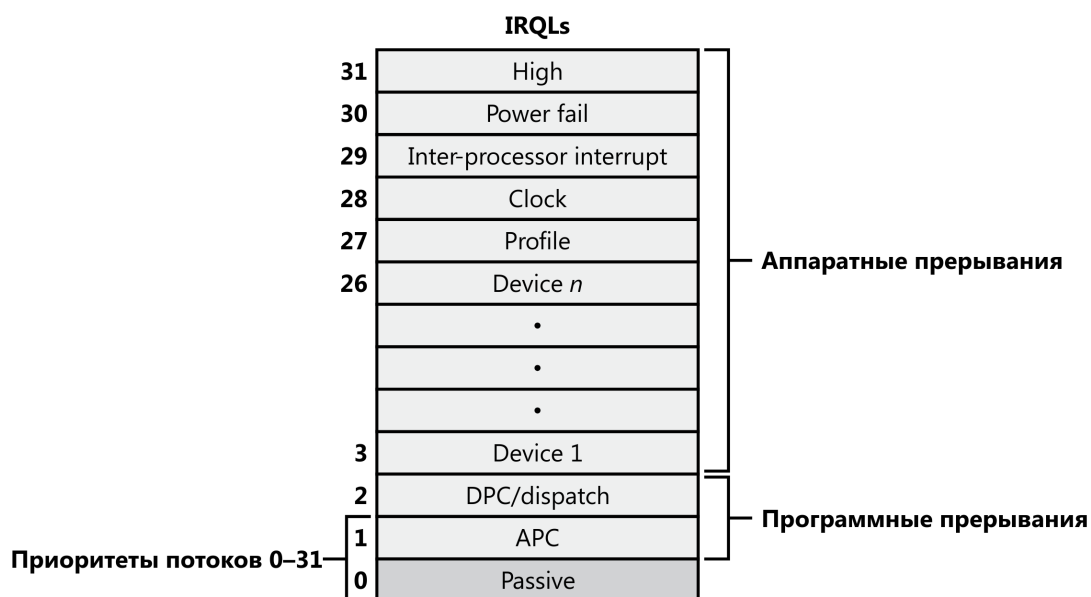


Рисунок 2.2 – Сопоставление приоритетов потоков с IRQL-уровнями на системе x86

Потоки обычно запускаются на уровне IRQL 0 или на уровне IRQL 1 (APC-уровень). Код пользовательского режима всегда запускается на пассивном уровне. Поэтому никакие потоки пользовательского уровня независимо от их приоритета не могут даже заблокировать аппаратные прерывания.

Потоки, запущенные в режиме ядра, несмотря на изначальное планирование на пассивном уровне или уровне APC, могут поднять IRQL на более высокие уровни, например, при выполнении системного вызова, который может включать в себя диспетчеризацию потока, диспетчеризацию памяти или ввод-вывод. Если поток поднимает IRQL на уровень dispatch или еще выше, на его процессоре не будет больше происходить ничего, относящегося к планированию потоков, пока уровень IRQL не будет опущен ниже уровня dispatch, так как прерывания обрабатываются в порядке из приоритетности. Поток выполняется на dispatch-уровне и выше, блокирует активность планировщика потоков и мешает контекстному переключению на своем процессоре.

ВЫВОДЫ

Оба семейства операционных систем Windows и UNIX являются системами разделения времени с динамическими приоритетами и вытеснением, поэтому обработчик прерывания от системного таймера выполняет в этих системах схожие функции:

1. выполняют декремент кванта;
2. инкрементирует счетчики времени и декрементирует счетчики таймеров, будильников реального времени, счетчики времени отложенных задач;
3. инициализирует отложенные действия, относящиеся к работе планировщика, такие как пересчет приоритетов.

Пересчет динамических приоритетов осуществляется только для пользовательских потоков для того, чтобы избежать их бесконечного откладывания. В ОС семейства UNIX приоритеты вычисляются по формуле, в которой учитываются время простоя в очереди готовых потоков и полученное процессорное время. В ОС семейства Windows пересчет приоритетов осуществляется путем сканирования очереди готовых потоков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Вахалия Ю.* UNIX изнутри. — 2003.
2. Внутреннее устройство Windows. 7-е изд. / М. Руссинович [и др.]. — Питер, 2022.